Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЁТ
к лабораторной работе №6
на тему
«ОСНОВЫ MONGODB»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Реализовать миграцию данных из формата *CSV* в хранилище *MongoDB*. Спроектировать структуру коллекций и выбрать подходящие типы данных.

2 Реализовать проверку целостности данных.

3 Реализовать получение заказов пользователя.

4 Реализовать получение детальной информации о заказе.

5 Реализовать получение топа товаров по выручке с использованием группировки и сортировки.

6 Провести проверку производительности запросов до и после создания индексов.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Хранилище MongoDB

MongoDB представляет собой документоориентированную систему управления базами данных класса *NoSQL*. Основной особенностью является хранение данных в виде JSON-подобных документов в формате *BSON*, что обеспечивает гибкость схемы и высокую масштабируемость.

2.2 Агрегационные Pipeline

Для выполнения сложных запросов в MongoDB используется фреймворк агрегации. Основные стадии:

- *$match* – фильтрация документов (аналог WHERE в SQL);
- *$group* – группировка документов по ключу (аналог GROUP BY);
- *$sort* – сортировка результатов (аналог ORDER BY);
- *$limit* – ограничение количества результатов;
- *$lookup* – соединение с другой коллекцией (аналог JOIN);
- *$unwind* – разворачивание массива в отдельные документы;
- *$project* – выбор конкретных полей (аналог SELECT).

2.3 Индексы

Индексы в MongoDB позволяют ускорить поиск документов. Применяются составные индексы, покрывающие несколько полей. Для анализа эффективности запросов используется метод *explain()* с параметром *executionStats*.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Проектирование схемы данных

Для хранения данных использовались следующие коллекции с соответствующими полями:

– *users* – { _id, name, email, createdAt };
– *categories* – { _id, name };
– *products* – { _id, name, categoryId, price };
– *orders* – { _id, userId, createdAt, status };
– *orderItems* – { _id, orderId, productId, quantity, price }.

Первичные ключи хранятся в поле *_id* для оптимизации доступа.

3.2 Миграция данных из CSV

Миграция осуществлялась путем последовательного чтения файлов CSV и записи документов в соответствующие коллекции. Числовые поля преобразовывались из строк в соответствующие типы (int, float64). Каждый документ содержит уникальный идентификатор в поле `_id`, который используется в качестве первичного ключа для быстрого доступа.

Ниже приведены примеры документов, которые создаются в коллекциях orders и orderItems в результате миграции.

Листинг 3.1 – Пример миграции заказов

```
db.orders.insertMany([
  { _id: 1, userId: 1, createdAt: "2024-03-15", status: "completed" },
  { _id: 2, userId: 1, createdAt: "2024-03-20", status: "shipped" },
  { _id: 3, userId: 3, createdAt: "2024-04-01", status: "processing" }
])
```

Листинг 3.2 – Пример миграции позиций заказов

```
db.orderItems.insertMany([
  { _id: 1, orderId: 1, productId: 5, quantity: 2, price: 1000.00 },
  { _id: 2, orderId: 1, productId: 3, quantity: 1, price: 500.00 },
  { _id: 3, orderId: 2, productId: 7, quantity: 3, price: 300.00 }
])
```

3.3 Проверка целостности данных

Для проверки ссылочной целостности используется агрегация с оператором *$lookup*. Этот запрос позволяет найти заказы, в которых поле userId ссылается на несуществующего пользователя. Результат показывает количество заказов с невалидными ссылками на пользователей.

Листинг 3.3 – Проверка userId в заказах

```
db.orders.aggregate([
  { $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user_details"
    }},
    { $match: { user_details: { $size: 0 } }},
    { $count: "invalid_orders_count" }
  ])
```

Результат показывает количество заказов с невалидными ссылками на пользователей.

Листинг 3.4 – Проверка общей стоимости заказа

```
db.orders.aggregate([
  { $addFields: {
    calculated_total: {
      $sum: {
        $map: {
          input: "$items",
          as: "item",
          in: { $multiply: ["$$item.quantity", "$$item.price"] }
        }
      }
    }
  }},
  { $match: { calculated_total: { $gt: 1000 } }},
  { $count: "expensive_orders_count" }
])
```

3.4 Получение заказов пользователя

Простой запрос для получения всех заказов конкретного пользователя. Фильтрация выполняется по полю userId, которое содержит идентификатор пользователя. Результаты не сортируются, порядок определяется внутренней структурой индекса.

Листинг 3.5 – GetUserOrders (простой Find)

```
db.orders.find({ userId: 1 })
```

3.5 Получение детальной информации о заказе

Для получения полной информации о заказе используется трехэтапное соединение коллекций. Сначала выбирается заказ, затем к нему присоединяются позиции заказа, и наконец к каждой позиции присоединяются данные о

товаре. Операция *$unwind* используется для разворачивания массива в отдельные документы, а *$group* собирает результат обратно в один документ.

Листинг 3.6 – GetOrdersWithDetails (3-way JOIN)

```
db.orders.aggregate([
  // Этап 1: выбор заказа по ID
  { $match: { _id: 1 }},

  // Этап 2: JOIN с orderItems
  { $lookup: {
    from: "orderItems",
    localField: "_id",
    foreignField: "orderId",
    as: "items"
  }},

  // Этап 3: разворачивание массива
  { $unwind: { path: "$items", preserveNullAndEmptyArrays: true }},

  // Этап 4: JOIN с products
  { $lookup: {
    from: "products",
    localField: "items.productId",
    foreignField: "_id",
    as: "items.product_details"
  }},

  // Этап 5: разворачивание деталей продукта
  { $unwind: { path: "$items.product_details",
preserveNullAndEmptyArrays: true }},

  // Этап 6: группировка обратно
  { $group: {
    _id: "$_id",
    userId: { $first: "$userId" },
    status: { $first: "$status" },
    createdAt: { $first: "$createdAt" },
    items: { $push: "$items" }
  }}
])
```

3.6 Получение топа товаров по выручке

Этот запрос вычисляет общую выручку для каждого товара. Сначала выполняется группировка по productId с вычислением суммы quantity * price. Затем результаты сортируются по убыванию выручки и ограничиваются топ-N позициями. Операция *$lookup* присоединяет названия товаров из коллекции products.

Листинг 3.7 – GetTopProductsByRevenue (GROUP BY)

```
db.orderItems.aggregate([
  // GROUP BY productId, SUM(quantity * price)
  { $group: {
    _id: "$productId",
    totalRevenue: { $sum: { $multiply: ["$quantity", "$price"] }},
    totalQuantity: { $sum: "$quantity" }
  }},

  // ORDER BY totalRevenue DESC
  { $sort: { totalRevenue: -1 }},

  // LIMIT N
  { $limit: 10},

  // JOIN для получения названий товаров
  { $lookup: {
    from: "products",
    localField: "_id",
    foreignField: "_id",
    as: "product_info"
  }},

  { $unwind: "$product_info" },

  // SELECT productId, name, revenue
  { $project: {
    productId: "$_id",
    name: "$product_info.name",
    revenue: "$totalRevenue",
    quantitySold: "$totalQuantity"
  }}
])
```

3.7 Проверка производительности

Для демонстрации влияния индексов на производительность выполнялось сравнение времени выполнения запроса до и после создания индекса с использованием метода *explain()*.

Листинг 3.8 – Создание составного индекса

```
db.orders.createIndex({ userId: 1, createdAt: -1 })
```

Листинг 3.9 – Получение статистики выполнения (MongoDB runCommand)

```
db.runCommand({
  explain: {
    find: "orders",
    filter: { userId: 1 },
    sort: { createdAt: -1 }
```

```
  },
  verbosity: "executionStats"
})
```

Результат содержит поля:

– *totalDocsExamined* – количество просмотренных документов;
– *nReturned* – количество возвращенных документов;
– *executionTimeMillis* – время выполнения в миллисекундах.

ВЫВОД

В ходе выполнения лабораторной работы были изучены принципы работы с документоориентированной СУБД *MongoDB*. Были приобретены следующие навыки:

– проектирование схемы данных с использованием коллекций и полей *_id* в качестве первичных ключей;

– реализация сложных запросов с использованием агрегационных *pipeline* с операторами *$lookup*, *$group*, *$unwind*

– выполнение соединений коллекций через *$lookup* для имитации реляционных JOIN;

– создание индексов для оптимизации запросов и измерение производительности.

Разработанное решение демонстрирует возможности NoSQL-хранилища MongoDB для обработки сложных аналитических запросов с использованием агрегационных пайплайнов.